

홍창희 · Backend Developer Portfolio

인증·인가와 데이터 정합성을 중심으로, 근거를 갖고 설명할 수 있는 백엔드 기능을 구현합니다.

ghdckdgm1191@gmail.com github.com/partmant

소개

인증/인가, 데이터 정합성, 문제 해결 과정을 근거로 설명할 수 있는 백엔드 개발자입니다. Java·Spring 프로젝트에서는 계정과 인증 도메인을 처음부터 끝까지 설계했고, Node.js 프로젝트에서는 ORM 없이 SQL을 직접 다루며 외부 데이터의 정합성 문제를 진단하고 해결했습니다. 팀 프로젝트에서는 본인이 실제로 구현한 범위와 팀 전체 성과를 구분해 설명하는 것을 기본 원칙으로 삼습니다.

핵심역량

- **인증·인가 설계** — Spring Security와 JWT 기반으로 Access/Refresh Token 발급, 역할별 API 접근 제어, 401·403 예외 응답을 구현. 필터 체인의 응답 생성과 실행 흐름 제어가 별개임을 실제 버그로 학습해, 검증 실패 시 체인 진행을 즉시 중단하는 구조까지 직접 다룸
- **데이터 정합성 및 모델링** — 외부 API와 내부 DB 간 코드 체계 불일치, 상태 전이에 따른 집계 오류를 데이터 모델과 집계 구조 개선으로 해결. 애플리케이션 순회 계산 대신 DB 집계 쿼리로 위임해 처리 순서와 무관한 역등 구조를 만드는 방식을 반복적으로 적용
- **SQL과 ORM 활용** — JPA·QueryDSL 기반 도메인 개발과 MySQL 윈도우 함수, 조인, Upsert 등 raw SQL 작성 경험 보유

보유 기술

- **Java / Spring Boot (중급)** — 4개 프로젝트에서 인증·인가, 데이터 집계, 상태 전이 등 주요 백엔드 도메인 로직 구현, REST API 설계
- **Spring Security / JWT (중급)** — 커스텀 JWT 인증 필터와 401·403 예외 처리 구현
- **JPA / QueryDSL (중급)** — 엔티티 설계·연관관계 매핑·낙관적 락(@Version) 적용, 동적 쿼리 및 집계 쿼리 작성
- **MySQL (중급)** — 스키마 설계, 윈도우 함수 기반 집계 쿼리 작성

활용 경험: Docker/GitHub Actions(초중급) · Node.js/Express(초급) · MyBatis/Neo4j/React(초급)

학력 · 교육이력

성공회대학교 · 정보통신공학(주전공) / 소프트웨어공학(부전공)

2020.03 ~ 2026.02 · 학점 3.68/4.5

이수 교과: 데이터베이스, 알고리즘, 자료구조, 운영체제, 백엔드 프레임워크, 네트워크 프로그래밍

멋쟁이사자처럼 · 백엔드 Java 부트캠프

2026.01 ~ 2026.07 · QuoteGuard, TILog, VolRank 팀 프로젝트를 진행한 교육 과정

자격증

- SQLD — 2025.04 취득 (2027.04 만료)
- 정보처리기사 — 2025.09 취득

프로젝트 요약

4개 프로젝트의 GitHub 커밋 이력과 담당 코드를 기준으로 집계했습니다.

프로젝트	주요 담당	담당 API	테스트 · 핵심 성과
QuoteGuard (대표)	인증 · 계정 · 통계 · CI/CD	20개	테스트 140건
TILog	작성이력 · 스트릭 · 구독 · 페이백	10개	테스트 7건 (보완 예정)
VolRank	지역연동 · 인증서 · 랭킹	12개	동기화 1분 → 10초 미만
FairMeeting	중간지점 계산 알고리즘	9개	예외 케이스 6종

전체 4개 프로젝트 기준 본인 커밋 276건(QuoteGuard 81, TILog 53, VolRank 59, FairMeeting 83), 담당 API 51개, 담당 테스트 147건입니다.

프로젝트 상세

QuoteGuard — B2B 견적 관리 시스템 (대표 프로젝트)

2026.06.10 ~ 07.08 (4 주) · 6인 팀(팀장) · 계정·인증·CI/CD

담당 API 20개 · 담당 테이블 4개 · 테스트 140건 · 커밋 81/700

영업 견적의 작성부터 승인, PDF 발송까지 처리하는 B2B 시스템에서, 계정 생성부터 로그인·권한 검증·배포까지 계정/인증 도메인 전체를 담당했습니다.

핵심 기능

- JWT Access/Refresh 토큰 기반 인증, 역할별 API 인가 (401/403 응답 분리)
- 이메일 기반 최초 비밀번호 설정 + 강제 변경 흐름
- 부서별·사용자별 통계 집계 기능
- Docker + GHCR + SSH 기반 CI/CD 파이프라인 구축

설계 의사결정 및 문제해결

문제 메뉴 숨김만으로는 API 직접 호출을 막지 못함

해결 SecurityConfig에서 역할 기반 접근 제어를 구성해 인증 실패(401)는 JwtAuthenticationEntryPoint, 인가 실패(403)는 JwtAccessDeniedHandler로 분리

문제 임시 비밀번호를 구두·메신저로 전달 시 유출 위험

해결 계정 생성 시 passwordInitialized=false로 시작, 이메일 링크로만 최초 비밀번호를 설정하게 하고 그 전에는 로그인 차단

문제 사용자 통계를 애플리케이션에서 전체 데이터를 순회해 계산하면 연산 부담이 커지고, 변경분만 더하고 빼는 방식은 상태 변경 순서에 따라 결과가 달라질 위험

해결 QueryDSL 기반 집계 쿼리(CASE-WHEN + SUM)로 건적 테이블 전체를 매번 다시 계산해, 상태 변경 이력의 순서에 의존하지 않고 현재 DB 상태를 기준으로 항상 다시 집계하도록 전환. @Version 낙관적 락으로 동시 갱신 충돌 처리

문제 인증 테스트 작성 중 무효한 JWT 토큰으로도 컨트롤러 로직 일부가 실행되는 것을 발견 — 필터에서 401 응답을 준비하는 코드는 있었지만 뒤에 return이 없어 filterChain.doFilter()가 그대로 실행되며 인증 필터 체인이 우회되고 있었음

해결 무효 토큰 분기 마지막에 return을 추가해 체인 진행 자체를 즉시 중단하도록 수정. 계정·인증 핵심 시나리오에 테스트 140건을 직접 작성해, 무효 토큰 요청이 모두 401로 종료되는지 확인

문제 로컬에서는 정상 동작하는 실시간 알림(SSE) 연결이 배포 환경에서만 주기적으로 끊기는 현상 발견 — 코드 자체에는 문제가 없어 원인 파악이 어려웠음

해결 연결 유지 시간과 배포 환경 리버스 프록시 설정을 비교 분석해 idle timeout이 원인임을 확인. 20초 주기로 더미 이벤트를 전송하는 하트비트를 추가해 연결이 끊기기 전에 계속 활성 상태로 유지되도록 수정

결과

- 인증 401과 403의 처리 책임을 분리하고 권한 검증을 서버 기준으로 일원화
- QueryDSL 집계 쿼리로 현재 DB 상태를 재계산해 상태 변경 순서에 따른 통계 불일치를 방지하는 구조 확보, 담당 도메인 테스트 140건 작성
- 배포 환경에서만 발생하던 SSE 연결 끊김을 하트비트로 해결해 실시간 알림을 안정화
- GitHub Actions로 빌드→GHCR 푸시→SSH 배포→컨테이너 재기동까지 7단계를 git push 1회로 자동 실행, 커밋 SHA 이미지 보관으로 이전 버전 재배포 가능한 기반 마련

TILog — 개발자 성장 기록 플랫폼

2026.05.28 ~ 06.07 (1.5 주) · 6인 팀(팀장) · 작성이력 스트릭·페이백

담당 API 10개 · 담당 테이블 5개 · 커밋 53/242

TIL 작성 이력을 스트릭과 잔디 히트맵으로 시각화하고, 꾸준한 학습에 보상을 연결하는 페이백 챌린지 기능을 백엔드부터 프론트엔드까지 담당했습니다.

핵심 기능

- 작성 이력을 게시글과 분리한 별도 도메인(WriteHistory) 설계
- 학습 스트릭 계산 및 잔디 히트맵 시각화
- 구독 취소 예약(CANCEL_RESERVED) 상태와 스케줄러
- 페이백 챌린지 참여·정책 도메인 설계

설계 의사결정 및 문제해결

문제 구독 취소 요청 즉시 프리미엄 권한을 제거하면 이미 확보한 이용 기간을 사용하지 못함

해결 취소 요청 시 상태를 ACTIVE → CANCEL_RESERVED로 전환, 매시 정각 스케줄러가 만료 대상을 스캔해 EXPIRED 처리 후 권한 복원

문제 페이백 진행률을 달력 기준(이번 달 1일~말일)으로 계산하고 있었는데, 실제 혜택이 적용되는 기간은 사용자의 실제 구독일 기준이라 두 기준이 어긋나 진행률이 실제와 다르게 표시되는 것을 QA 중 발견

해결 참여 데이터에 실제 시작일·종료일과 그 시점의 정책 스냅샷을 함께 저장하도록 구조를 바꿔, 진행률 계산이 달력이 아니라 저장된 실제 기간을 기준으로 이뤄지도록 수정

결과

- 작성이력·스트릭·구독·페이백 도메인에서 API 10개, 테이블 5개를 설계·구현
- 짧은 개발 기간으로 구독·페이백 테스트를 충분히 작성하지 못했으며, 상태 전이와 스케줄러 중복 실행을 중심으로 테스트를 보완할 계획

VolRank — 봉사활동 랭킹·인증서 관리 서비스

2026.03.26 ~ 04.10 (2주) · 6인 팀 · 지역연동 인증서·랭킹

담당API 12개 · 담당테이블 3개 · 동기화 1분→10초 미만 · 커밋 59/217

봉사활동 인증서 검토와 지역 데이터 정합성 관리, 랭킹 집계 SQL을 ORM 없이 raw SQL 기반으로 직접 설계·구현했습니다.

핵심 기능

- RANK() 윈도우 함수 기반 전국·지역 랭킹 집계
- 지역 코드 2단계 계층(parent_code 자기참조) 매핑
- 1365 공공 API 연동 및 봉사공고 수집
- 관리자 인가(isAdmin) 미들웨어 구현

설계 의사결정 및 문제해결

문제 1365 자원봉사포털 API 응답 필드명이 코드의 매핑과 달라, 봉사 활동 시작·종료 시각이 잘못 계산되어 저장되고 있는 것을 발견. 계산된 값만 저장하는 구조라 원본 데이터가 없어 사후 복구도 어려웠음

해결 필드명을 실제 응답에 맞게 수정하고, 계산된 값 대신 시작·종료 시각 원본 컬럼을 분리해 저장하도록 데이터 모델을 변경해 이후 재계산이 가능하도록 함

문제 봉사 실적 등록 API(POST /register-volunteer)에 로그인 인증 미들웨어가 빠져 있어, 인증 없이도 호출이 가능했고 비로그인 상태에서는 사용자 식별 값이 없어 서버 오류로 이어짐

해결 해당 라우트에 인증 미들웨어를 추가해 로그인하지 않은 요청은 실행 자체가 차단되도록 수정하고, 다른 라우트도 인증 미들웨어 적용 여부를 전수 점검

문제 지역 동기화를 페이지 단위로 순차 호출해 전체 동기화에 최대 1분 가까이 소요

해결 첫 응답에서 전체 데이터 수를 확인해 필요한 페이지 수를 계산한 후, 남은 페이지를 3개씩 청크로 나눠 Promise.all로 동시 호출하도록 전환. 거의 변하지 않는 지역 코드 데이터는 메모리에 캐싱해 반복 조회도 제거

결과

- 외부 데이터 소스의 정합성 문제를 코드 레벨에서 진단하고 해결하는 경험 확보
- 인증 없는 API 호출을 원천 차단하고 관련 라우트를 전수 재점검
- 전체 동기화 시간을 최대 약 1분에서 10초 미만으로 단축 (실측)
- 랭킹은 애플리케이션 순회 정렬 대신 RANK() SQL 윈도우 함수로 DB에서 집계·정렬하도록 설계해 사용자 수 증가에도 애플리케이션 서버 부하가 늘지 않는 구조 확보

FairMeeting — 이동시간 편차 기반 중간지점 추천 (캡스톤)

2025.03.10 ~ 06.15 (캡스톤) · 3인 팀(팀장) · 알고리즘 예외 처리 UI 공동 구현

담당API 9개 · 예외 케이스 6종 · 커밋 83/192 (3인 중 최대)

여러 출발지의 이동 시간 편차를 최소화하는 중간지점을 그래프 DB 기반 데이터로 계산하고, 6종의 경계값 예외를 처리하는 비즈니스 로직을 설계했습니다.

핵심 기능

- 다중 출발지 다익스트라 + N-1 부분집합 최적화 알고리즘 직접 구현
- 그래프 데이터베이스(Neo4j)에 저장된 지하철 노선 데이터를 조회해 알고리즘 입력으로 활용
- 출발지 개수·거리·서비스 범위 등 6종 경계값 예외 처리
- 위치·경로·카테고리 검색 API 설계

설계 의사결정 및 문제해결

문제 출발지 중 하나가 계산된 중간지점과 정확히 일치하는 경우처럼, 일부 또는 전체 출발지의 소요 시간이 동시에 0분으로 계산되는 경계 조건에서 편차 비교 로직이 무의미해지거나 예상치 못한 결과를 반환할 위험 발견

해결 경계 상황을 유형별로 분류해, 소요시간 0분은 정상 처리로 허용하되 계산 자체가 무의미해지는 조건에서는 명시적으로 예외를 발생시켜 잘못된 결과가 그대로 반환되지 않도록 처리

결과

- 검색·경로·중간지점 관련 API 9개 구현, 경계 상황 6종을 정의해 각각 다른 처리 방식(차단/제한/정상 처리)을 코드 레벨에서 구분